

Practical Lab – 06

Topic: Django CRUD Tutorial- Operations and Application Development

Introduction:

Django has an amazing framework. Now, it's time to make a web application using newly gained skills. We will discuss some important concepts that are required to understand while making an application.

In this lab session, we will develop a library application to perform the CRUD operations (Create, Read, Update & Delete) in a Django app.

Aims of the lab:

By the end of this lab, students will be able to:

- Understand the main concepts of database which are CRUD operations
- Perform CRUD operations in a Django app

Tools: VS Code, python, Django

Prerequisites:

Setting Up Django in VS Code:

So far, these are the **full and quick** steps for *Setting Up Django in VS Code*:

First, **create folder** in your system and **open it in VS Code**, then open your terminal and do the following:

Step 1: Create a virtual environment

```
py -3 -m venv .venv (Windows) || python3 -m venv .venv (Mac)
```

Note: Make sure to select the virtual environment from (*View > Command Palette*) then [*Python: Select Interpreter*], then select the one with *./.venv* or *.\.venv* (Recommended)

Step 2: Activate it

```
.venv\Scripts\activate (Windows) || source .venv/bin/activate (Mac)
```

Note: if you have any problem with activation, run this command in your *PowerShell*:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Step 3: Install Django

```
python -m pip install django
```

Step 4: Create a new Django project

```
django-admin startproject webproject
```

Step 5: Move into your project directory

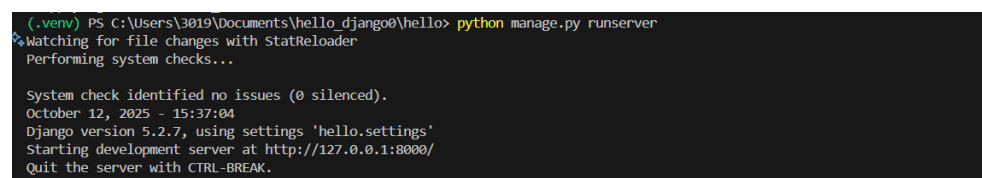
```
cd webproject
```

Step 6: Create an empty development database:

```
python manage.py migrate
```

Step 7: Run the development server

```
python manage.py runserver
```

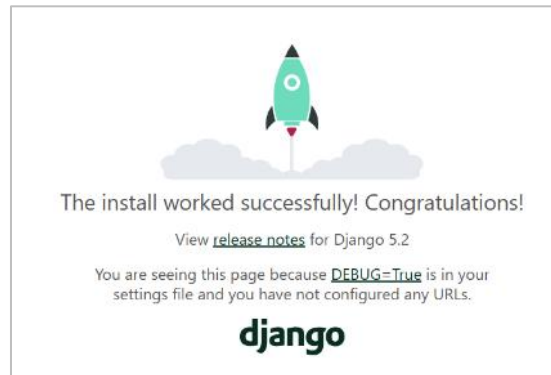


```
(.venv) PS C:\Users\3019\Documents\hello_django0\hello> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
October 12, 2025 - 15:37:04
Django version 5.2.7, using settings 'hello.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

>> Open <http://127.0.0.1:8000> in your browser to see the admin interface.

>> To stop the server in VS Code use: (**Ctrl+C**).



Note: Before starting to work on your project, make sure your virtual environment is **activated**. If not, activate your environment, using:

```
.venv\Scripts\Activate.ps1 (Windows) || source .venv/bin/activate (MacOS).
```

Part 1: An Overview of CRUD operations:

What is CRUD?

CRUD stands for **Create, Read, Update & Delete**. These are the four basic operations which are executed on **Database Models**. We are developing a **web app** which can perform these operations.

Since we are developing a library app, let's take an example of the same. In a library, **books** are **objects**. The **books** have **attributes** like **name, author**, etc. We need an application which can perform **CRUD operations** on **book objects**. The CRUD operations are defined as follows:

1. **Read Operation:** The ability of the application to **read data from the database**.
2. **Create Operation:** The ability of the application to **store data in the database**.
3. **Update Operation:** The ability of the application to **edit the stored value in the database**.

4. **Delete Operation:** The ability of the application to **delete the value in the database**.

We are going to develop the operations in the same order.

Majority of applications on the internet are CRUD applications. **For example—Facebook** uses CRUD operations to **save your data on their database**. You can **change your profile picture** that means perform the **update** operation. Of course, you can **see the data in-app or browser** which is **read** operation.

Also, you can **delete your Facebook account** which is **delete** operation. Summarizing it, almost all the applications you use are **CRUD applications**.

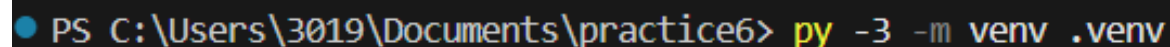
For developers, making a CRUD application is one of the very first steps. If you can make a CRUD app from technology, then you can start deploying projects.

Part2: Django CRUD Tutorial

Let's explore the steps to design a CRUD application in Django framework:

1. Creating a Library CRUD Application

Our first steps would be to do a new project in Django and **create a new virtual environment**. To make a virtual environment, enter:



```
PS C:\Users\3019\Documents\practice6> py -3 -m venv .venv
```

Figure 1 virtualenv library(command)

Now, install the necessary software in this environment. Once the environment is set up, activate the same. After **activating the environment, install Django on it**.

```

• PS C:\Users\3019\Documents\practice6> .venv\Scripts\activate
• (.venv) PS C:\Users\3019\Documents\practice6> python -m pip install django
Collecting django
  Using cached django-5.2.7-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref<=3.8.1 (from django)
  Using cached asgiref-3.10.0-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse<=0.3.1 (from django)
  Using cached sqlparse-0.5.3-py3-none-any.whl.metadata (3.9 kB)
Collecting tzdata (from django)
  Using cached tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Using cached django-5.2.7-py3-none-any.whl (8.3 MB)
Using cached asgiref-3.10.0-py3-none-any.whl (24 kB)
Using cached sqlparse-0.5.3-py3-none-any.whl (44 kB)
Using cached tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.10.0 django-5.2.7 sqlparse-0.5.3 tzdata-2025.2

[notice] A new release of pip is available: 24.3.1 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
○ (.venv) PS C:\Users\3019\Documents\practice6> 

```

Figure 2: *pip install django==2.1.7(command)*

After the installation is completed, **create a new Django project.**

>> *django-admin startproject libraryapp*

```

(.venv) PS C:\Users\3019\Documents\practice6> django-admin startproject libraryapp

```

Figure 3: *django-admin startapp libraryapp (command)*

```

▼ PRACTICE6
  > .venv
  > libraryapp

```

The project is made by Django. Now, the last command is to make a new application. **Move to the directory of manage.py file and make a new app.**

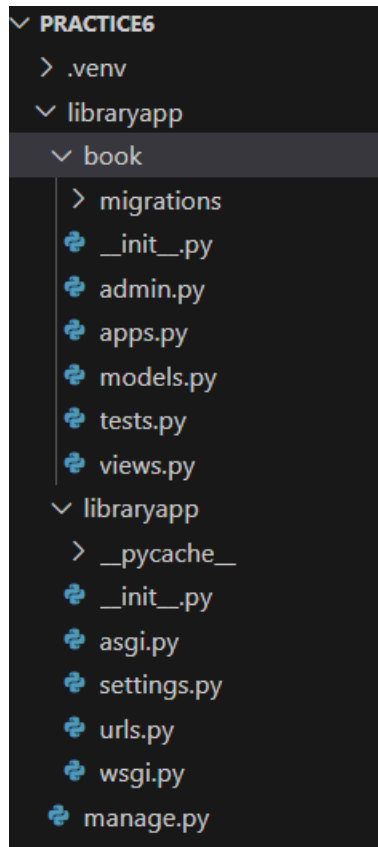
```

(.venv) PS C:\Users\3019\Documents\practice6> cd ./libraryapp/
(.venv) PS C:\Users\3019\Documents\practice6\libraryapp> python manage.py startapp book

```

Figure 4: *python manage.py startapp book (command)*

Now, all the required setups are complete. We can proceed further.



2. Installing Application and Other Important Settings

To install the app, just add the application's name in the `INSTALLED_APPS` list. This is inside **settings.py** file.

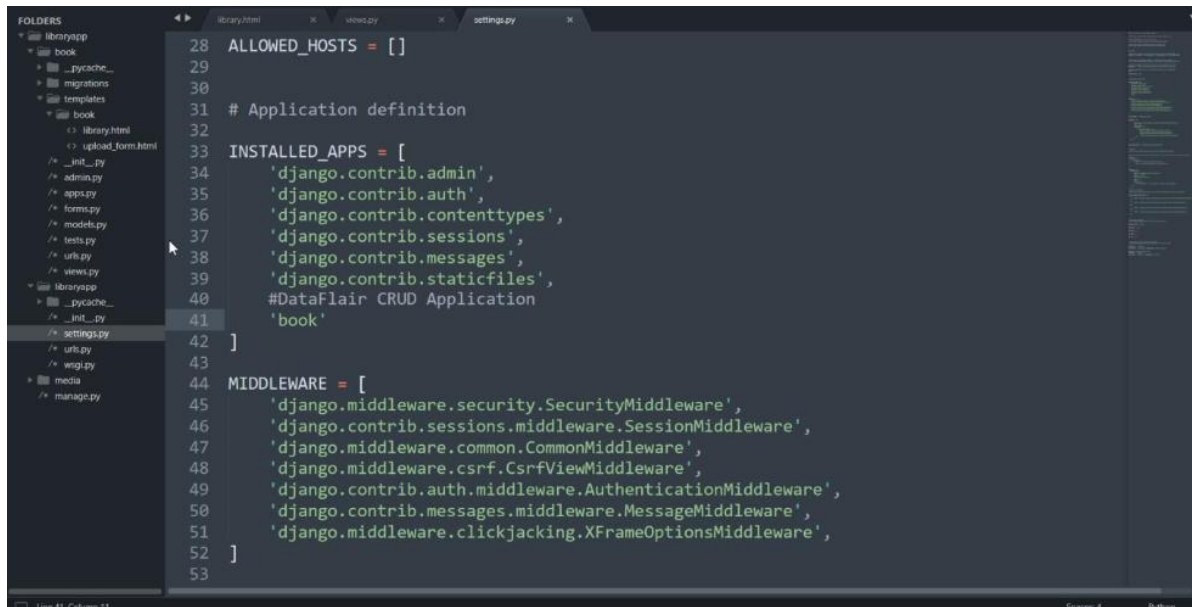


Figure 5: settings.py

Once you complete this step, we also need to **establish the Static Files Settings**. **Copy these settings** as it is and replace them with the existing settings of your project.

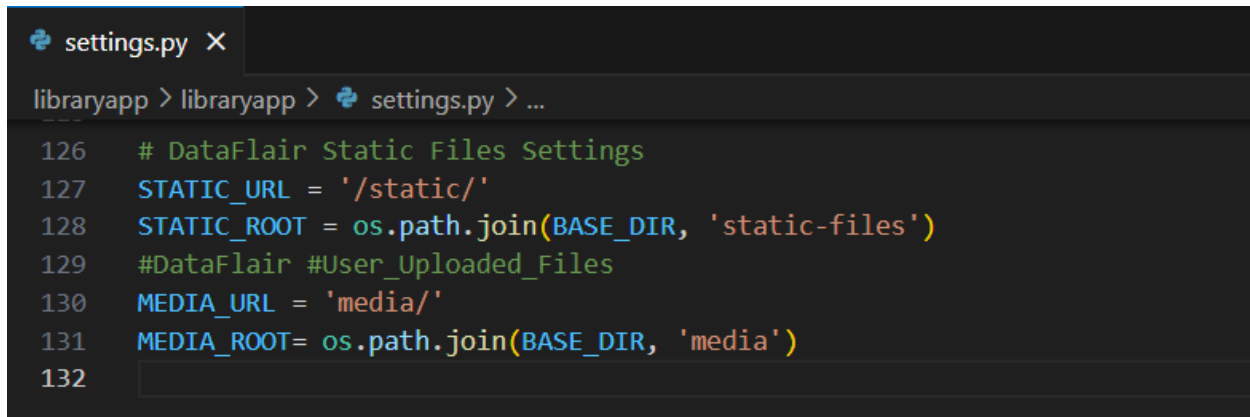
Code:

```
# DataFlair Static Files Settings
STATIC_URL = '/static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'static-files')
#DataFlair #User_Uploaded_Files
MEDIA_URL = 'media/'
MEDIA_ROOT= os.path.join(BASE_DIR, 'media')
```

Note: you need to import **os** when you use **os.path.join()** in your Django settings file. In the beginning of *settings.py*, write this:

```
import os
```

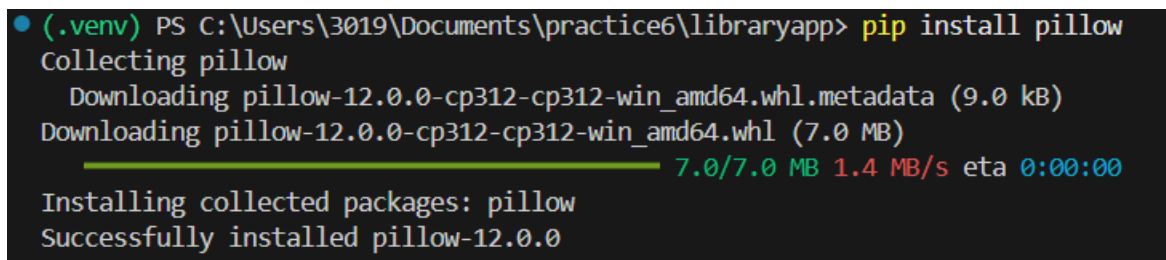
Code Display:



```
settings.py X
libraryapp > libraryapp > settings.py > ...
126 # DataFlair Static Files Settings
127 STATIC_URL = '/static/'
128 STATIC_ROOT = os.path.join(BASE_DIR, 'static-files')
129 #DataFlair #User_Uploaded_Files
130 MEDIA_URL = 'media/'
131 MEDIA_ROOT= os.path.join(BASE_DIR, 'media')
132
```

Figure 6: settings.py

Above setting is **important for saving and serving static files to the client**. Also, we **need a pillow package of Python**. Install it **via command line** in your virtual environment.



```
(.venv) PS C:\Users\3019\Documents\practice6\libraryapp> pip install pillow
Collecting pillow
  Downloading pillow-12.0.0-cp312-cp312-win_amd64.whl.metadata (9.0 kB)
  Downloading pillow-12.0.0-cp312-cp312-win_amd64.whl (7.0 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.0/7.0 MB 1.4 MB/s eta 0:00:00
Installing collected packages: pillow
Successfully installed pillow-12.0.0
```

Figure 7: pip install pillow (command)

Pillow is a **Python Imaging Library** which deals with **different image files**. We won't be **using pillow directly**, but **Django is using it**. Therefore, we need pillow library.

Okay, now all the **preparations for the application are complete**. All the **settings are done** so we can start developing the same.

3. Creating Models for Books App

In the book folder, open *models.py* and paste this code. We are **creating a model or database table for our books app**.

Code:

```
from django.db import models

#DataFlair Models
class Book(models.Model):
    name = models.CharField(max_length = 50)
    picture = models.ImageField()
    author = models.CharField(max_length = 30, default='anonymous')
    email = models.EmailField(blank = True)
    describe = models.TextField(default = 'DataFlair Django tutorials')

def __str__(self):
    return self.name
```

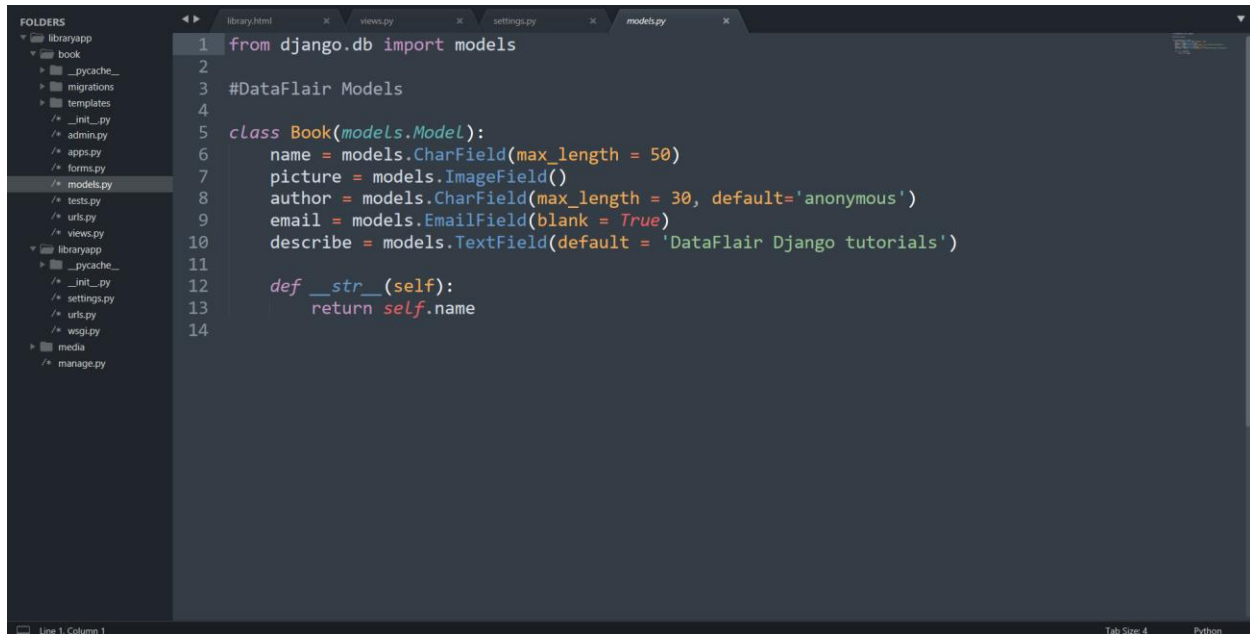
Code Display:

Figure 8: models.py

Understanding the Code:

From the above code, **Django** will make a table in a database named **book**. That table will have these fields:

Name, picture, author, email, and description of the **book object**. From this class, we can easily **manipulate the table**. The important part here is **the picture field**. It is set as *ImageField*. This *ImageField* is the reason why we installed **pillow** in this virtual environment. Django **by default uses pillow to handle Images** in *ImageField*. So, that was **the models of our book app**.

4. Making Model Forms in Book Directory

Django makes it so much easier to **make forms for models**. We just need to use our models, and we can easily make forms. Create a new file **forms.py** in *book directory*. Paste this code in this *forms.py*.

Code:

```
from django import forms
from .models import Book
#DataFlair
class BookCreate(forms.ModelForm):
    class Meta:
        model = Book
        fields = '__all__'
```

Code Display:

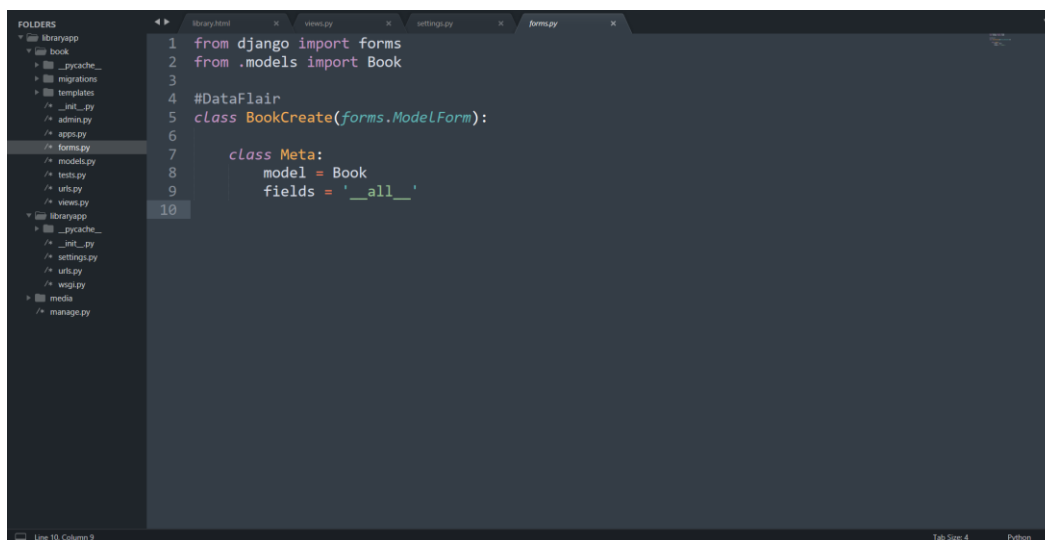


Figure 9: forms.py

Understanding the Code:

This is a **simple model form** we created for **book model**. Here we **import two libraries/modules**, namely, *forms* and *Book*. The *Book* is the *model* we just created.

The *BookCreate()* class is the **form class** which **represents the Model form**. In the *MetaClass*, we **define which model to be used to create the model form**.

Fields denote the **actual form fields** where we want **input from the user**. This form is enough for our work.

5. Registering Model in Django Admin

Here we are editing *admin.py* existing in **book folder**. **Import the model** you want to **register in the admin**. In this case, it is a *Book*. Then paste this line below it.

Code:

```
from .models import Book

#DataFlair
admin.site.register(Book)
```

Code Display:

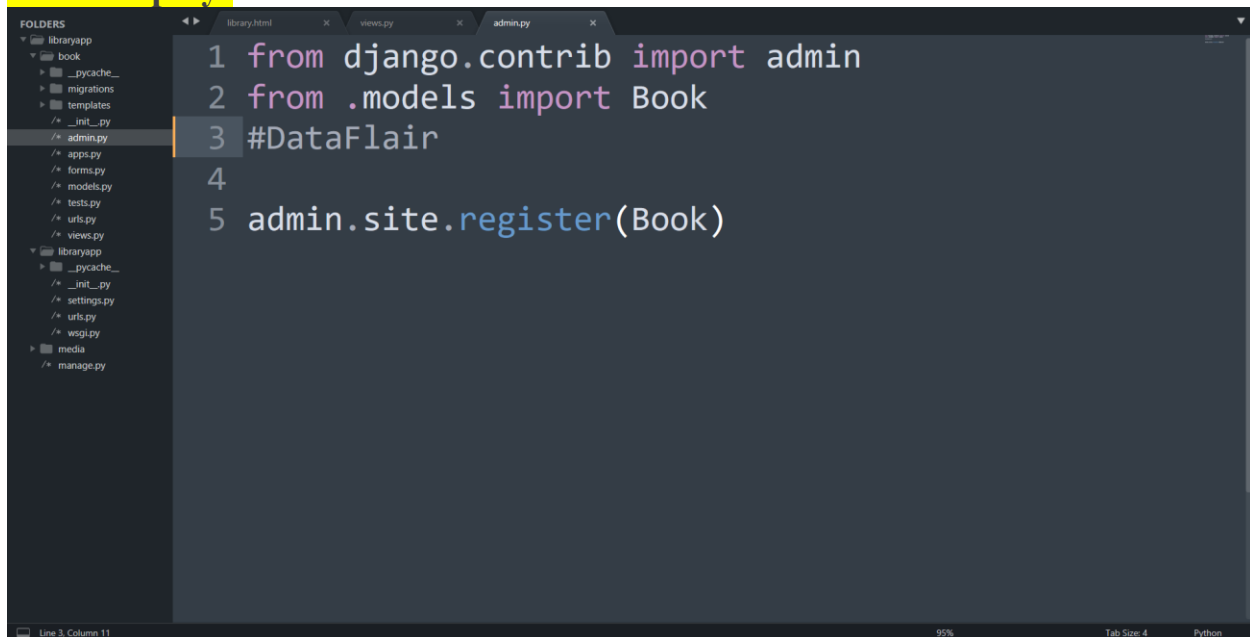


Figure 10: admin.py

Now, when we have described a *Model* in the *models.py* file, we must run some commands to **create the table in the database**. Thus, to **implement all of this**, run these commands in the command line:

python manage.py makemigrations
python manage.py migrate

```
● (.venv) PS C:\Users\3019\Documents\practice6\libraryapp> python manage.py makemigrations
Migrations for 'book':
  book\migrations\0001_initial.py
    + Create model Book
● (.venv) PS C:\Users\3019\Documents\practice6\libraryapp> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, book, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying book.0001_initial... OK
  Applying sessions.0001_initial... OK
○ (.venv) PS C:\Users\3019\Documents\practice6\libraryapp> 
```

Figure 11: Create database in Django (command)

Note: If you are following the **Django CRUD tutorial**, there will be no errors. If something does arise then tell us.

After running these commands, we also need a **superuser** to login to the admin. You can **create a superuser** using the command:

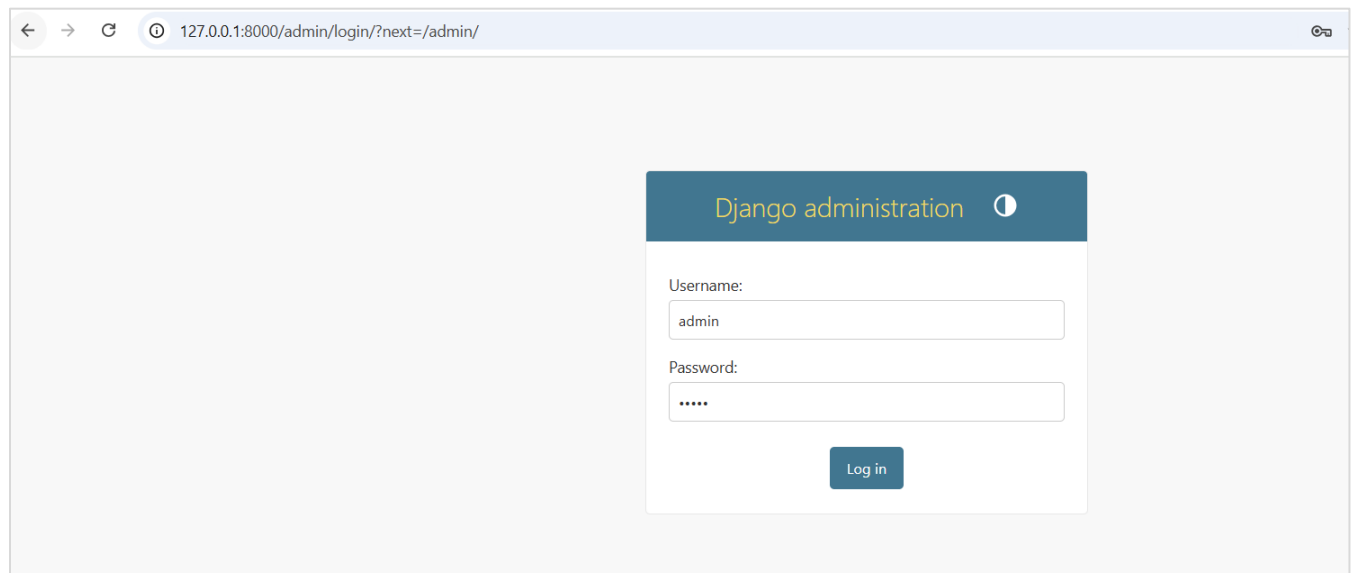
```
(.venv) PS C:\Users\3019\Documents\practice6\libraryapp> python manage.py createsuperuser
Username (leave blank to use ): admin
Email address: admin@gcet.edu.om
Password:
Password (again):
The password is too similar to the username.
This password is too short. It must contain at least 8 characters.
This password is too common.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
(.venv) PS C:\Users\3019\Documents\practice6\libraryapp> █
```

Figure 12: `python manage.py createsuperuser` (command)

After creating a **superuser**, you can access the **Django admin control panel** by starting the development server first:

`python manage.py runserver`

Go to <http://127.0.0.1:8000/admin/>, log in with your **superuser credentials**, and you'll see the **Django Admin Dashboard**.



Now, we have **created models and forms files** so far. Now, let's make the *views* for our **CRUD application**.

6. Making View Functions for Django CRUD App

The **view functions** are our **actual CRUD operations in Django**. Now, we are editing *views.py* in *book folder*. Open *views.py* file in the folder. Paste this code in it:

Code:

```
from django.shortcuts import render, redirect, get_object_or_404
from .models import Book
from .forms import BookCreate
from django.http import HttpResponseRedirect

#DataFlair
#List all books.
def index(request):
    shelf = Book.objects.all()
    return render(request, 'book/library.html', {'shelf': shelf})

#Create a new book (handles GET and POST)..
def upload(request):
    upload = BookCreate()
    if request.method == 'POST':
        upload = BookCreate(request.POST, request.FILES)
        if upload.is_valid():
            upload.save()
            return redirect('index')
        else:
            return HttpResponseRedirect("""your form is wrong, reload on <a href = "{url : 'index'}" >reload</a>""")
    else:
        return render(request, 'book/upload_form.html', {'upload_form':upload})

#Update an existing book identified by book_id...
def update_book(request, book_id):
    # Get the book or return a 404 if it doesn't exist
    book = get_object_or_404(Book, id=book_id)
```

```
if request.method == 'POST':
    # Bind form with POST data and files
    form = BookCreate(request.POST, request.FILES, instance=book)
    if form.is_valid():
        form.save()
        return redirect('index')
else:
    # On GET, show the form pre-filled with the book's data
    form = BookCreate(instance=book)

    # Render the form template (if GET or invalid form)
    return render(request, 'book/upload_form.html', {'upload_form': form, 'book':
book})

#Delete an existing book identified by book_id...
def delete_book(request, book_id):
    try:
        book_sel = Book.objects.get(id=book_id)
    except Book.DoesNotExist:
        # If no book found, just go back to index
        return redirect('index')

    # Delete the found book
    book_sel.delete()

    # Redirect to index after deletion
    return redirect('index')
```

```

libraryapp > book > views.py > update_book
1 from django.shortcuts import render, redirect
2 from .models import Book
3 from .forms import BookCreate
4 from django.http import HttpResponse
5
6 #DataFlair
7 #List all books.
8 def index(request):
9     shelf = Book.objects.all()
10    return render(request, 'book/library.html', {'shelf': shelf})
11
12 #Create a new book (handles GET and POST)..
13 def upload(request):
14     upload = BookCreate()
15     if request.method == 'POST':
16         upload = BookCreate(request.POST, request.FILES)
17         if upload.is_valid():
18             upload.save()
19             return redirect('index')
20         else:
21             return HttpResponseRedirect("your form is wrong, reload on <a href = '{{ url : 'index'}}'>reload</a>")
22     else:
23         return render(request, 'book/upload_form.html', {'upload_form':upload})
24

```

```

libraryapp > book > views.py > update_book
23 return render(request, 'book/upload_form.html', {'upload_form':upload})
24
25 #Update an existing book identified by book_id...
26 def update_book(request, book_id):
27     # Get the book or return a 404 if it doesn't exist
28     book = get_object_or_404(Book, id=book_id)
29
30     if request.method == 'POST':
31         # Bind form with POST data and files
32         form = BookCreate(request.POST, request.FILES, instance=book)
33         if form.is_valid():
34             form.save()
35             return redirect('index')
36     else:
37         # On GET, show the form pre-filled with the book's data
38         form = BookCreate(instance=book)
39
40     # Render the form template (if GET or invalid form)
41     return render(request, 'book/upload_form.html', {'upload_form': form, 'book': book})
42
43
44 #Delete an existing book identified by book_id...
45 def delete_book(request, book_id):
46     try:
47         book_sel = Book.objects.get(id=book_id)
48     except Book.DoesNotExist:
49         # If no book found, just go back to index
50         return redirect('index')
51     # Delete the found book
52     book_sel.delete()
53     # Redirect to index after deletion
54     return redirect('index')
55

```

Figure 14: views.py

Understand the Code:

There is a lot of information in this file. *views.py* is one of the **main backend components** and is the **business logic layer**. So, let's understand it step by step.

In this file, we have imported **Book Model** and **BookCreate Form**.

1. Index Function

This function is performing **Read Operation**. In this function, we **simply retrieve all the objects on the book table**. Those objects are then **passed to the corresponding template**.

We are using **Querysets** here for that purpose. As discussed in previous sessions: **Querysets is used to retrieve data from Tables**. There are all kinds of **filters** and **usage of Querysets** and here we are using:

Book.objects.all()

It is clear from the **query** that it is passing a set of **all objects in Book Table**.

2. Upload Function

This function is **CREATE operation of CRUD**. It is simply taking **the data entered from the user (using the form)** and **saving it in a database**. Since we created a **model** for that, we don't need to **validate data again**. We can **directly save the form info in the database**.

We first **create the form object**. Then we **check whether the form is submitting data** or user is visiting for the first time.

If the form **request method** is **POST**, it means the form is being **submitted**. You can see, it is also checked whether the form also has an image file or not.

The ***request.FILES*** is a **dictionary**-like object containing the **FILES** and other information.

Then we check **whether the data entered by the user is correct or not**. This is done by **form_object.is_clean()** method. This will return **True or False** whether the **form_object** holds valid data or not.

If the answer is **True**, we save the form data received in the database. **form_object.save()** accomplishes this and since it's a model form, we can use it directly.

If we receive a **GET request**, then we return an **empty form**. That's how we can create an object in the database.

Note: No distinction between GET and POST

- The function always tries to handle **request.POST**, even when the user first opens the edit page (GET).
- It should show the form first (GET) and only process data when the user submits it (POST).

3. Update_book Function

The **update_book Function** is a bit like the **Update** Function. It does more than that though. The **update_book** function takes **two parameters from the request**. The **request itself and id number**. The **id number** is used to **identify the object** which is to be edited.

You can pass it in as a **URL or as a cookie**. The **session method** is the most secure, but we don't need to use it here. So, the **update_book** function will check whether the **book_id** is valid or not.

If the **object exists** it will **return the form filled with the object's information** in it. The user can change the form again. In this case, there will be no creation of new books but the editing of the existing book object.

4. Delete Function

Delete Function is the last function of the CRUD application. We are again using the **same object method** as with **Update book function**. We are passing the **request** and **book_id** to delete the book.

Note: This is a simpler interpretation of update_book function.

The *queryset book.objects.get (id = book_id)* will check for the books having an id equal to book_id. Since book_id is a **primary key**, we will have **only one object returned**. We can **delete that object** easily by just executing:

***Book.delete()* method.** This will delete the book from the database.

So, these were the **view functions**. Now, we are ready to make the **templates** and complete our app.

7. Making Templates

The first thing you need to do is to make the templates folder in the *book folder*. Inside *book/templates*, make *another folder book*.

We are going to make all our templates in that folder. Inside *book/templates/book*, make a new file *library.html*:

Code:

```
<!DOCTYPE html>
{% load static %}
<html>
<head>
<title>Books App</title>
<link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css"
integrity="sha384-
ggOyR0iXCbmQv3Xipma34MD+dH/1fQ784/j6cY/iJTQU0hcWr7x9JvoRXT2MZw1T"
crossorigin="anonymous">
</head>
<body style="background-color:#4DB6AC;">
<nav class="navbar navbar-light" style="background-color:#FF0033;">
<a href="#" class="navbar-brand" style="background-
color:#FFCC33;color:#000000;border-width:10px;border-color:#FFCC33;border-
style:solid;padding:6px;border-radius:5px;">Library App</a>
<a class="navbar-link btn-lg" href="{% url 'index' %}" style="background-
color:#FFCC33;color:#000000;">DataFlair</a>
<a href="{% url 'admin:index' %}" class="navbar-link btn-lg" style="background-
color:#FFCC33;color:#000000;">Admin</a>
<a href="{% url 'upload-book' %}" class="navbar-link btn-lg" style="background-
color:#FFCC33;color:#000000;">Upload Books</a>
</nav>
<br>
{% block content %}
<div class="card-columns">
{% for book in shelf %}
<div class="card" style="width: 18rem;">

<div class="card-body">
<h5 class="card-title">{{ book.name }}</h5>
<p class="card-text">{{ book.describe }}</p>
<div class="card-footer bg-transparent border-dark">
<p class="card-title">{{ book.author }}</p>
<center>
<a href="update/{{book.id}}" class="btn btn-warning" id = '{{book.id}}'>edit</a>
<a href="delete/{{book.id}}" class="btn btn-danger" id = '{{book.id}}'>delete</a>
</center>
</div>
</div>
</div>
</div>
```

```

</div>
</div>
</div>
{% endfor %}
</div>
{% endblock %}
</body>
</html>

```

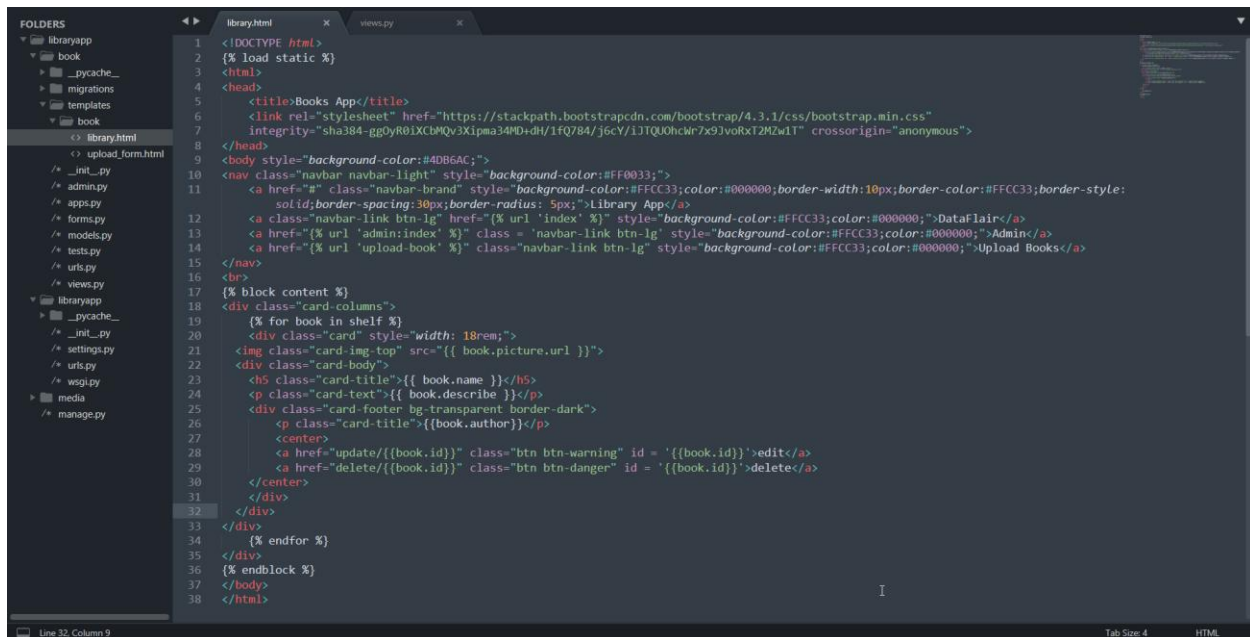


Figure 15: library.html

Understanding the Code:

It's a simple template file where we are displaying objects from a database. We are running a for loop to access the data in the dictionary we passed. All the other things are CSS and some Bootstrap Framework.

Code:

```

{% extends 'book/library.html' %}
{% block content %}
<center>

```

```

<h1 class="display-3" style="background-color:#000000;color:#FFFF99;">Upload
Books</h1>
  <form method = 'POST' enctype="multipart/form-data">
{% csrf_token %}
<table class = 'w-50 table table-light' style="border-radius:10px;background-
color:#FFFF99;">
{% for field in upload_form %}
<tr>
<th>{{field.label}}</th>
<td>{{ field }}</td>
</tr>
{% endfor %}
</table>
<button type="submit" class="btn btn-lg btn-warning">Submit</button>
</form>
</center>
{% endblock %}

```

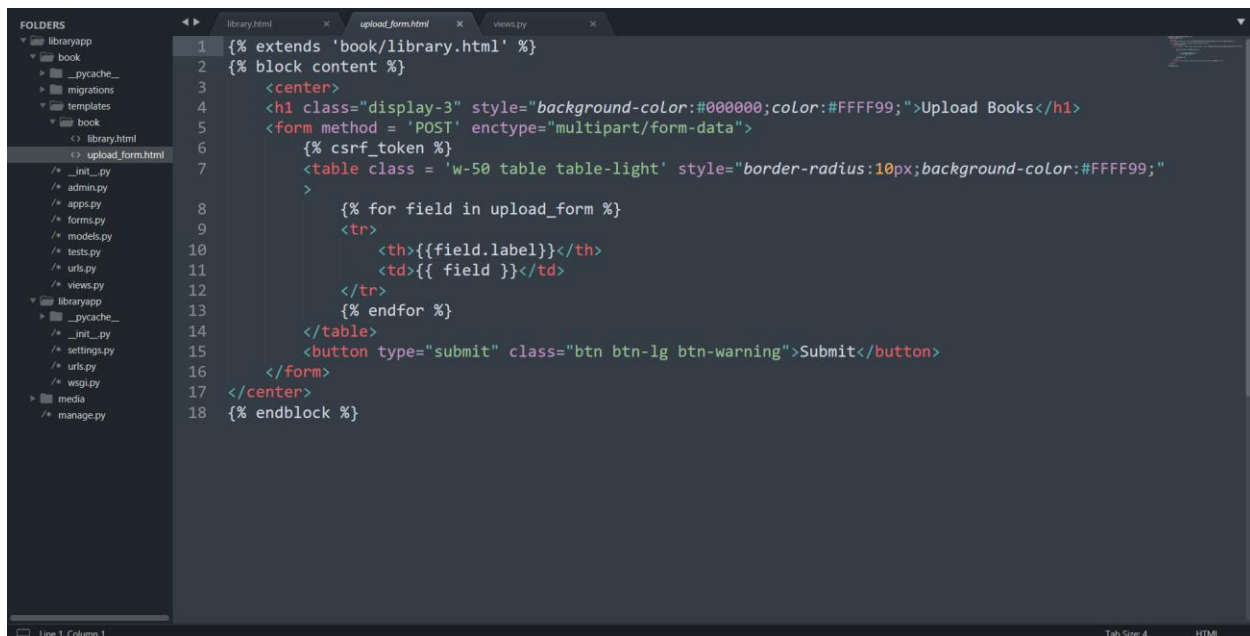


Figure 16: Upload_form.html

Understanding the Code:

This is a typical form rendering template. We are using csrf token and other form tags. Here I am printing the form fields via for loop. This can be done in another way directly. It depends on how you want to render it in frontend.

8. Configuring URLs

Now, we need to configure the *urls* file. Paste this code as it is in the mentioned *urls* files. If the file doesn't exist, then make one and copy the whole thing.

Code:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('book.urls'))
]
```

Code Display:

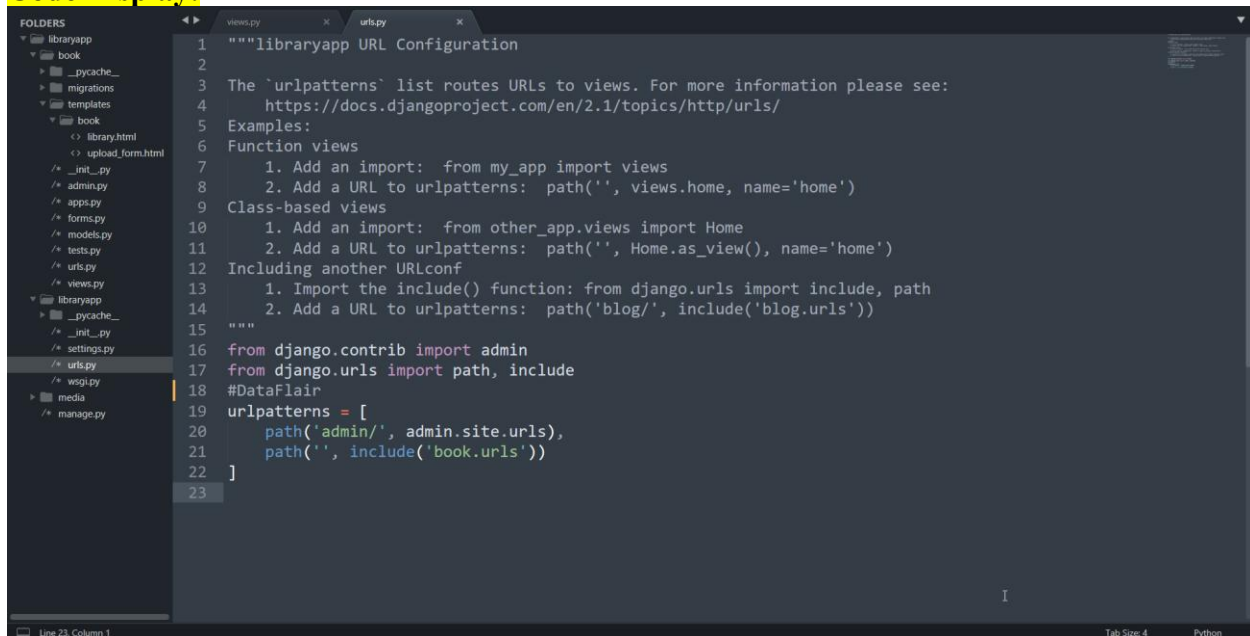


Figure 17: Urls.py

Urls.py in book folder:

Code:

```
from django.urls import path
from . import views
from libraryapp.settings import DEBUG, STATIC_URL, STATIC_ROOT, MEDIA_URL,
MEDIA_ROOT
from django.conf.urls.static import static
urlpatterns = [
    path('', views.index, name = 'index'),
    path('upload/', views.upload, name = 'upload-book'),
    path('update/<int:book_id>', views.update_book),
    path('delete/<int:book_id>', views.delete_book)
]
#DataFlair
if DEBUG:
    urlpatterns += static(STATIC_URL, document_root = STATIC_ROOT)
    urlpatterns += static(MEDIA_URL, document_root = MEDIA_ROOT)
```

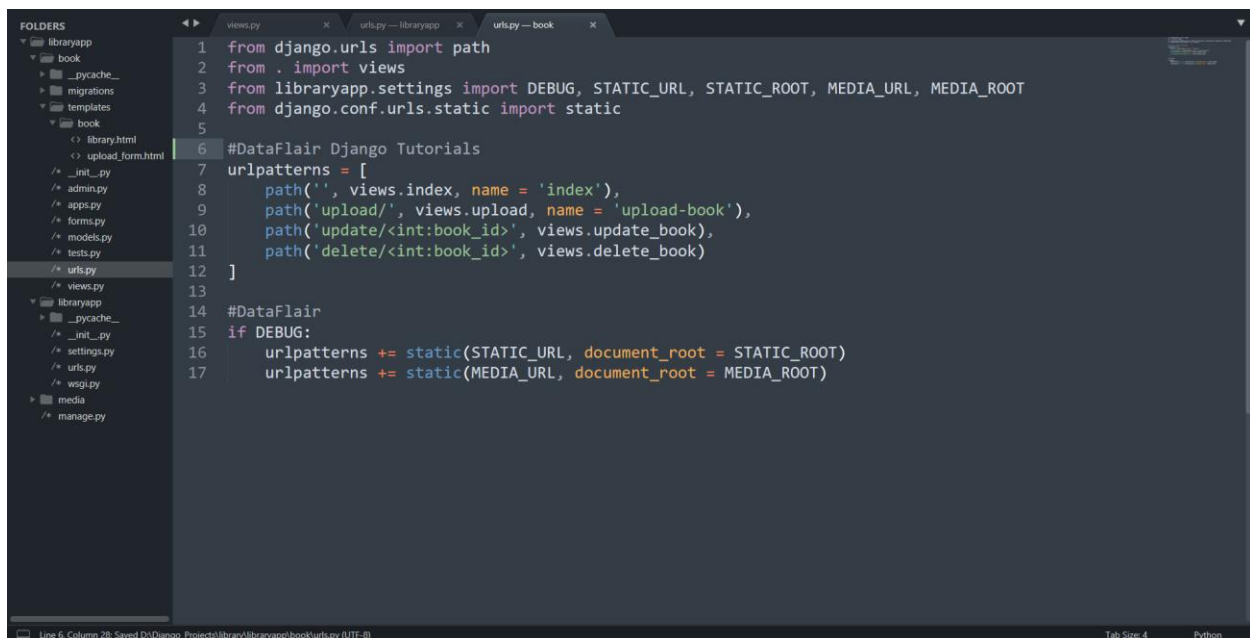


Figure 18: Book/urls.py

This file contains all the URL lookups for library app as well as settings for Django static files. These settings are used by Django to render the images with the book object.

9. Running Server and Testing

At last, the fun part. To test the website, start your server by:

```
(.venv) PS C:\Users\3019\Documents\practice6\libraryapp> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
November 02, 2025 - 21:59:29
Django version 5.2.7, using settings 'libraryapp.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

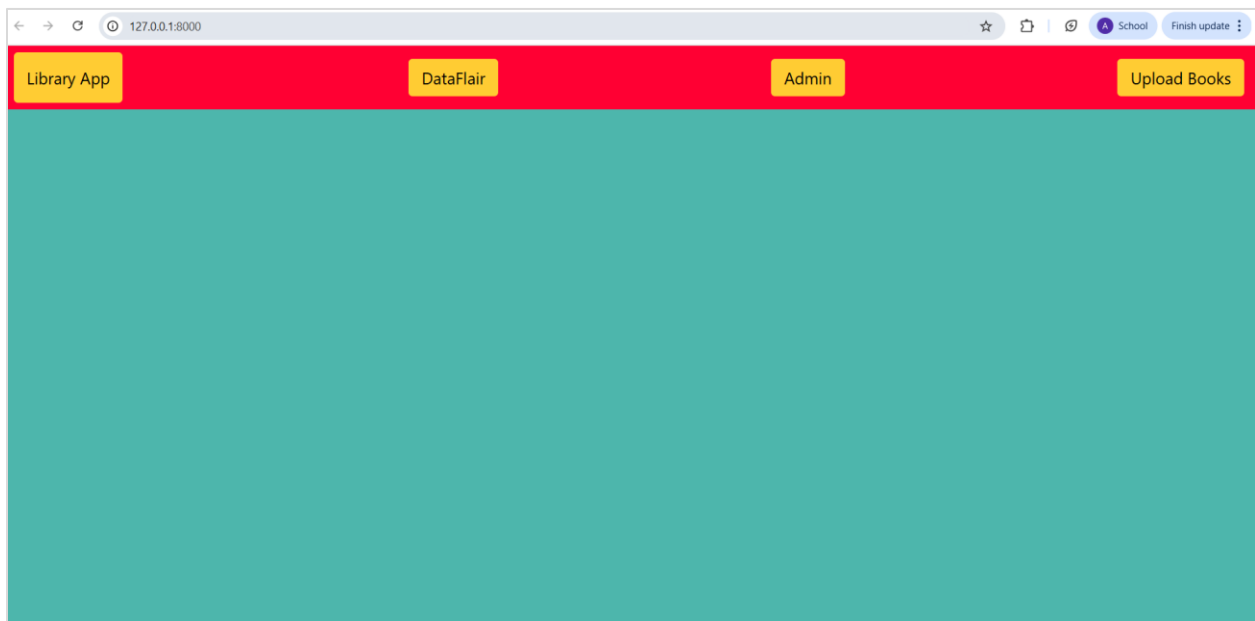
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[02/Nov/2025 21:59:33] "GET / HTTP/1.1" 200 956
```

Figure 19: python manage.py runserver (command)

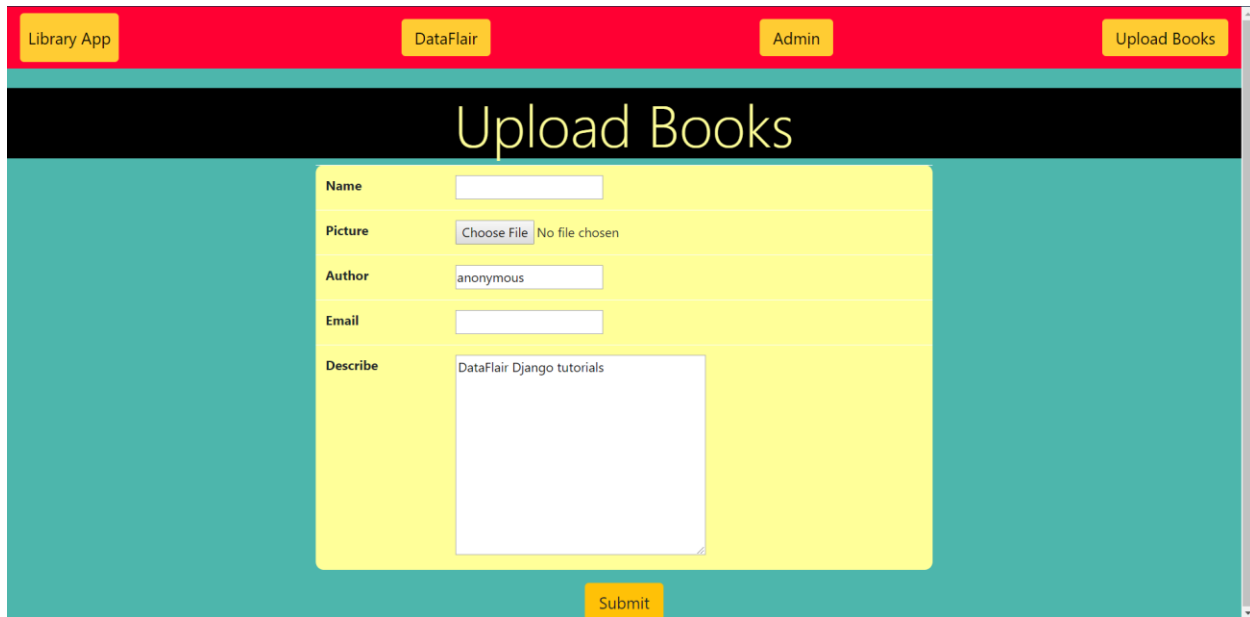
Now, open the **URL**:

<http://127.0.0.1:8000/>

You can see a page like this.



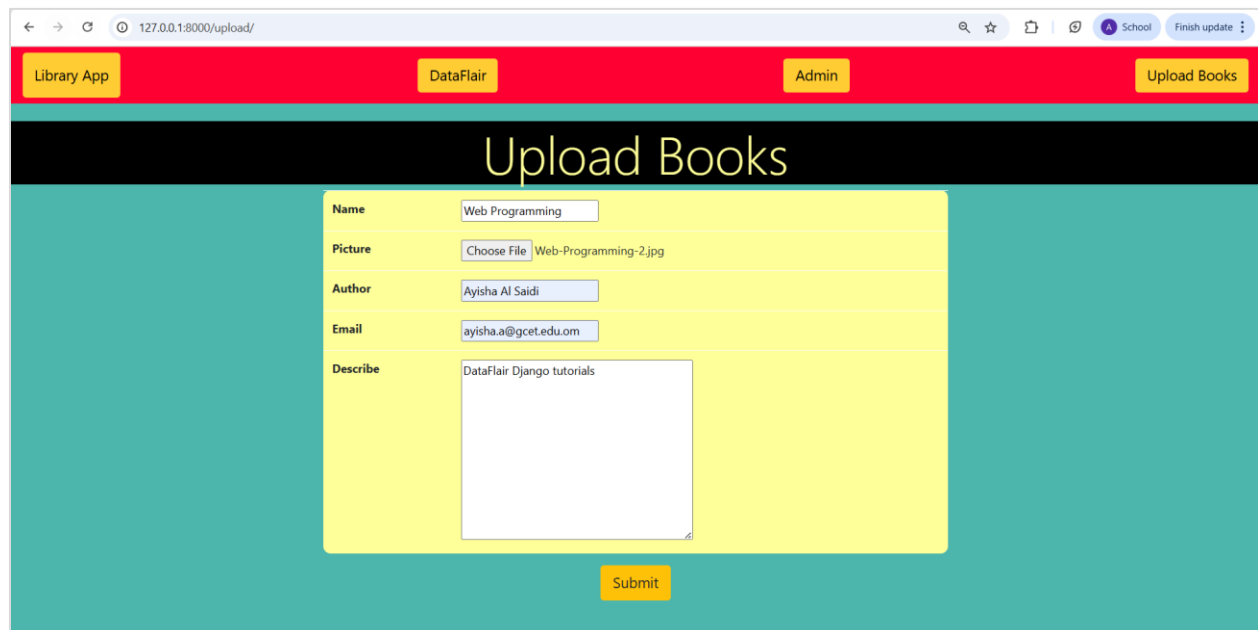
Click on **Upload book** link on the top-right corner of the screen.



The screenshot shows a web application interface with a red header bar containing four yellow buttons: 'Library App', 'DataFlair', 'Admin', and 'Upload Books'. Below the header is a black banner with the text 'Upload Books' in yellow. The main content area has a teal background. In the center is a yellow form with the following fields: 'Name' (empty text input), 'Picture' (file upload button labeled 'Choose File' and 'No file chosen'), 'Author' (text input with 'anonymous'), 'Email' (empty text input), and 'Describe' (text area with 'DataFlair Django tutorials'). A yellow 'Submit' button is at the bottom of the form.

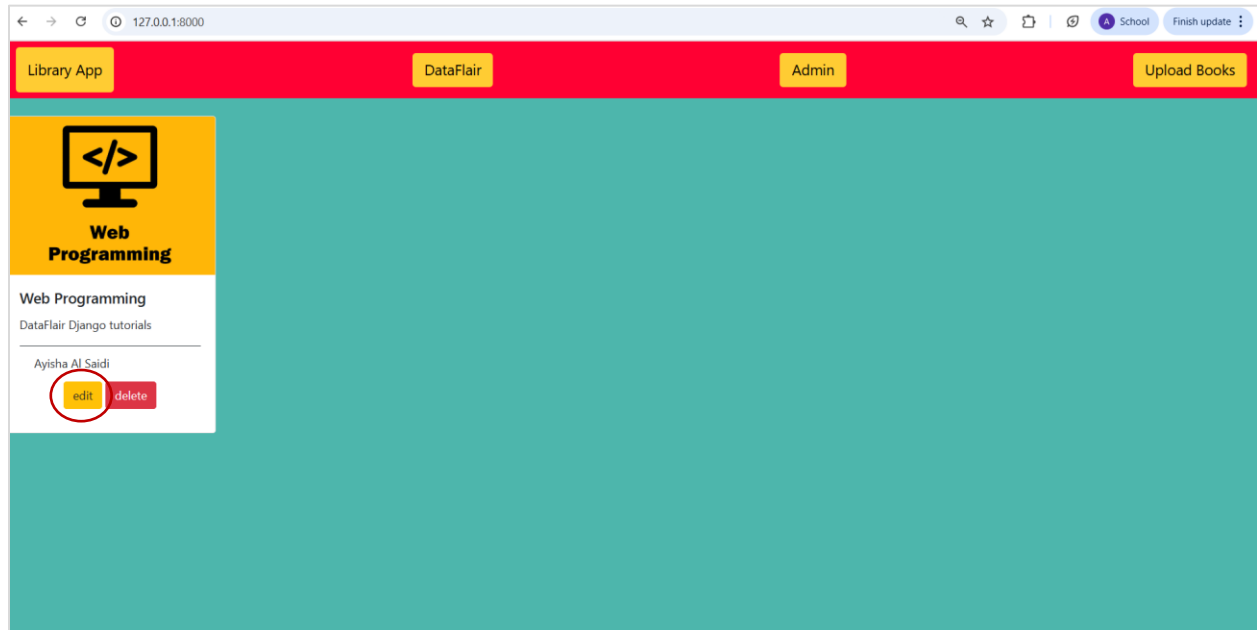
Figure 20: <http://127.0.0.1:8000/>

As you will see, we will have a form filled with books information.

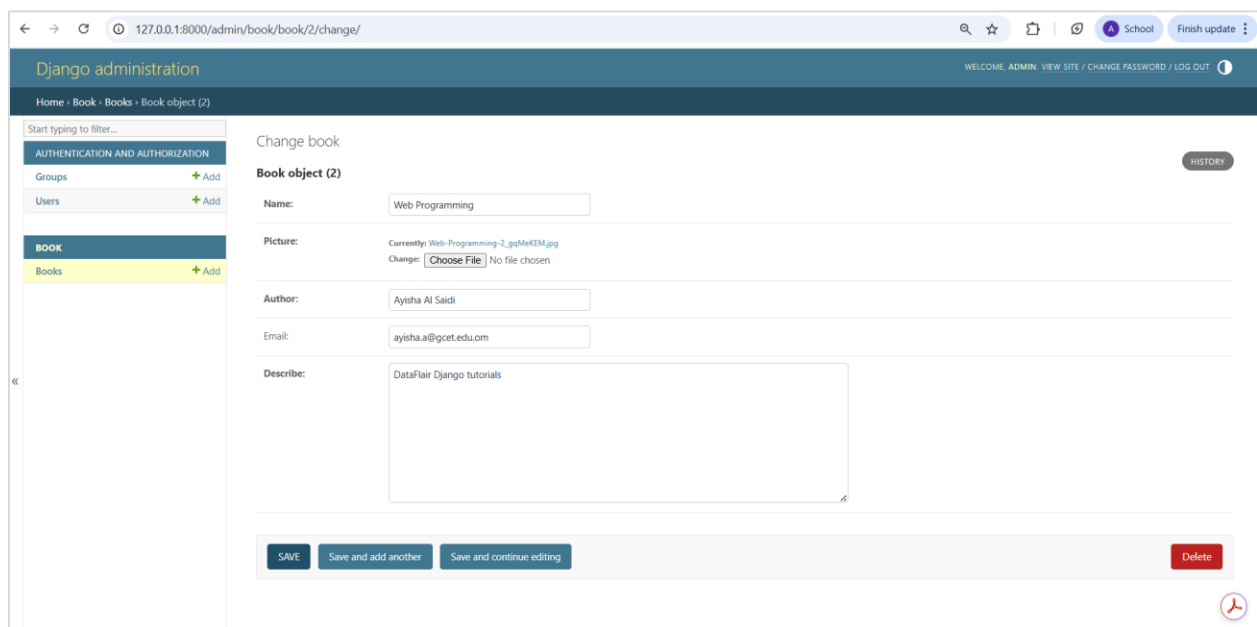


This screenshot shows the same 'Upload Books' form as Figure 20, but with pre-filled data. The browser's address bar shows '127.0.0.1:8000/upload/'. The form fields are now: 'Name' (filled with 'Web Programming'), 'Picture' (filled with 'Web-Programming-2.jpg'), 'Author' (filled with 'Ayisha Al Saidi'), 'Email' (filled with 'ayisha.a@gcet.edu.om'), and 'Describe' (filled with 'DataFlair Django tutorials'). The 'Submit' button remains at the bottom.

Figure 21: <http://127.0.0.1:8000/>



Now, there is only one book in the database. All these changes are occurring at the level of the database. You can see it in the admin dashboard:



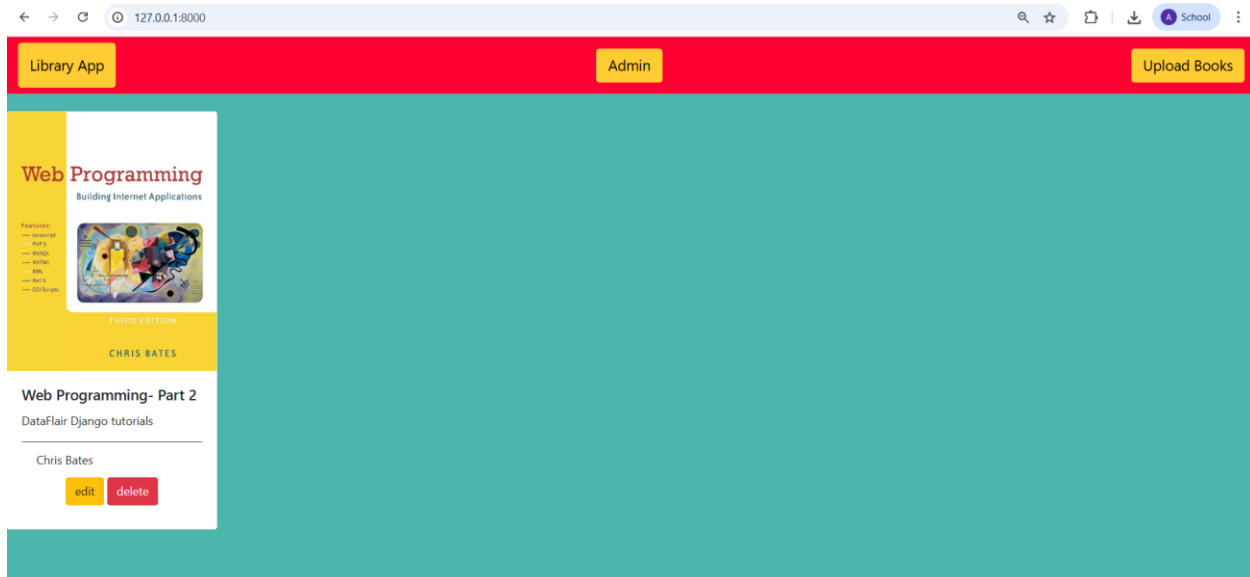
Use Edit:

You can now **change the information accordingly** and submit it again by clicking on ***'edit' button***. This will change the data in the database.

The screenshot shows a web browser at the address 127.0.0.1:8000/update/3. The page has a red header with three buttons: 'Library App', 'Admin', and 'Upload Books'. The main content area has a teal background with a black banner at the top that says 'Upload Books' in yellow. Below the banner is a yellow form with the following fields:

- Name:** Web Programming
- Picture:** Currently: Web-Programming-2_DoXp11.jpg. Change: Choose File No file chosen
- Author:** Ayisha Al Saidi
- Email:** ayisha.a@gcet.edu.om
- Describe:** DataFlair Django tutorials

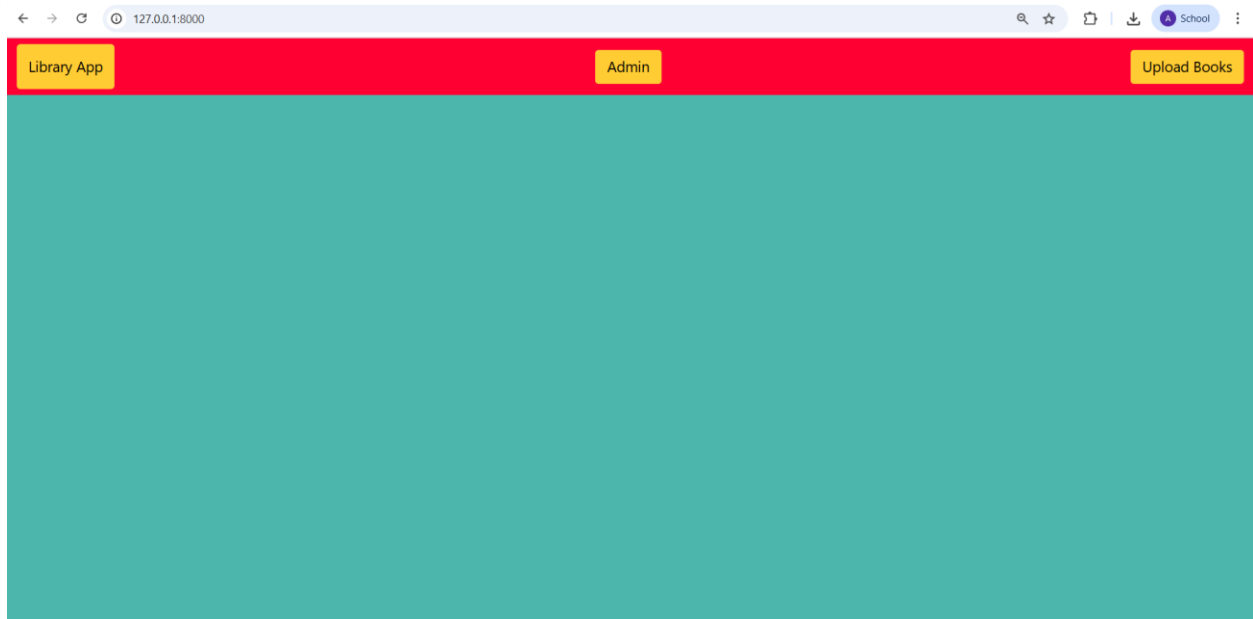
A yellow 'Submit' button is located at the bottom right of the form.



Now the data is updated.

Use Delete:

Lastly, when you click the **deleted button** in a book object, the book will be deleted from the database. **Note:** the book is deleted as you can see below.



Summary

Lastly, you have a working prototype of a **library app**. Enhance it more or make something new using the basic CRUD operations in Django. The CRUD operations make it much easier for you to implement website solutions.

Practical Lab – 07

Topic: Django Web Application Development - Login and Logout for user

Introduction:

In this lab session, we will focus on developing the library application by creating the Login, Dashboard, and Logout pages within the Django Library Project and the library app.

Aims of the lab:

In this lesson, you will learn how to:

- Create users
- Build a login page
- Authenticate users
- Redirect to a Dashboard after login
- Protect the dashboard with login restrictions
- Implement logout and redirect back to Home
- Add URLs, views, and templates

Tools: VS Code, python, Django

Creating Login, Dashboard, and Logout Pages in a Django Library Project

This guide assumes you already have created **library** project with the **libraryapp** in the previous lab.

However, here is a complete step-by-step setup for your **library web application**.

1. Creating Users in Django project

For a successful login, we need to have authenticated users, so we must create users inside Django.

Step 1: Open Django Shell

```
(.venv) PS C:\Users\3019\Documents\practice6\libraryapp> python manage.py shell
```

```
>> python manage.py shell
```

Step 2: Create a User

Inside the shell type:

```
from django.contrib.auth.models import User
User.objects.create_user(username='ayisha', password='ayisha123')
```

Note: put your username / password.

You can create as many users as you want.

```
>>> from django.contrib.auth.models import User
>>> User.objects.create_user(username='ayisha', password='ayisha123')
>>> User.objects.create_user(username='ayisha', password='ayisha123')
<User: ayisha>
>>> ^Z

now exiting InteractiveConsole...
```

Step 3: Exit

Press: **Ctrl + Z >> then press Enter**

Your user is now successfully created.

2. Creating Dashboard Page

Now, to create a page (Dashboard Page) that displays after a user logs in to your Django project, we need to do the following:

1. Setting Up Static Files (if not done)

In `settings.py`:

```
126 # Library Static Files Settings
127 STATIC_URL = '/static/'
128 STATIC_ROOT = os.path.join(BASE_DIR, 'static-files')
```

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
```

This allows your project to load any static files like CSS files.

2. Creating the Dashboard Page Template

Create `dashboard.html` in the template file:

```
libraryapp/templates/book/dashboard.html
```

dashboard.html

```
<!DOCTYPE html>
{% load static %}
<html>
<head>
    <title>Library App - Dashboard</title>

    <!-- Bootstrap -->
    <link rel="stylesheet"
        href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.
min.css"
```



```

        integrity="sha384-
gg0yR0iXCBMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T"
        crossorigin="anonymous">

</head>

<body style="background-color:#4DB6AC;">

<!-- NAVBAR -->
<nav class="navbar navbar-light" style="background-color:#FF0033;">
    <a href="#" class="navbar-brand"
        style="background-color:#FFCC33;color:#000000;border-width:10px;
        border-color:#FFCC33;border-style:solid;padding:6px;border-
radius:5px;">
        Library App
    </a>

    <!--<a href="{% url 'admin:index' %}" class="navbar-link btn-lg"
        style="background-color:#FFCC33;color:#000000;">Admin</a>-->

    <!-- <a href="{% url 'upload-book' %}" class="navbar-link btn-lg"
        style="background-color:#FFCC33;color:#000000;">Upload Books</a>-->

    <a href="{% url 'dashboard' %}" class="navbar-link btn-lg"
        style="background-color:#FFCC33;color:#000000;">Dashboard</a>

    <a href="{% url 'logout' %}" class="navbar-link btn-lg"
        style="background-color:#FFCC33;color:#000000;">Logout</a>
</nav>

<br><br>

<!-- DASHBOARD CONTENT-->
<div class="container">
    <div class="row justify-content-center">
        <div class="col-md-7">

            <div class="card shadow-lg">
                <div class="card-header text-center" style="background-
color:#FFCC33;">
                    <h3>Welcome to Your Dashboard</h3>
                </div>

                <div class="card-body text-center">

```

```
<h4>Hello, <b>{{ user.username }}</b> </h4> <!-- Personalized
greeting -->

<p>You have successfully logged in to the Library App.</p>

<br>

<!-- Dashboard Actions -->
<div class="list-group">
  <a href="{% url 'upload-book' %}"
    class="list-group-item list-group-item-action">
    Upload a New Book
  </a>

  <a href="{% url 'index' %}"
    class="list-group-item list-group-item-action">
    View All Books
  </a>

  <a href="{% url 'admin:index' %}"
    class="list-group-item list-group-item-action">
    Admin Panel
  </a>
```

Notes:

- ✓ This page only appears when the user successfully logs in.
- ✓ User must be authenticated to access it.

3. Creating the Dashboard View (Protected by Login)

Now, Add a view function to render the *dashboard.html* template in:

libraryapp/views.py

Add this code:

```
from django.contrib.auth.decorators import login_required

@login_required
```

```
def dashboard(request):  
    return render(request, 'book/dashboard.html')
```

```
from django.contrib.auth.decorators import login_required
```

```
@login_required  
def dashboard(request):  
    return render(request, 'book/dashboard.html')
```

Note: Why use `@login_required` decorator?

This decorator ensures that only authenticated users can access the dashboard page, thus:

- Prevents non-logged-in users from opening `/dashboard/`
- Automatically redirects them to the login page

Now, we need to Redirect the user to the Dashboard After Login successfully.

4. Creating the Login View to

To redirect users to the dashboard after a successful login, we need to update the login view in `views.py`, so add the following code:

```
from django.shortcuts import render, redirect  
from django.contrib.auth.decorators import login_required  
from django.contrib.auth import authenticate, login, logout  
  
@login_required  
def dashboard(request):  
    return render(request, 'book/dashboard.html')  
  
def login_view(request):  
    if request.method == 'POST':  
        username = request.POST['username']  
        password = request.POST['password']  
  
        user = authenticate(request, username=username, password=password)
```

```
if user is not None:
    login(request, user)
    return redirect('dashboard') # Redirect after login
else:
    return render(request, 'book/login.html',
                  {'error': 'Invalid username or password'})

return render(request, 'book/login.html')
```

6. Creating the Login Page Template: (login.html)

Create:

libraryapp/templates/book/login.html

login.html

```
<!DOCTYPE html>
{% load static %}
<html>
<head>
    <title>Library App - Login</title>

    <!-- Bootstrap -->
    <link rel="stylesheet"
        href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.
min.css"
        integrity="sha384-
ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T"
        crossorigin="anonymous">

</head>

<body style="background-color:#4DB6AC;">

<!-- NAVIGATION BAR -->
<nav class="navbar navbar-light" style="background-color:#FF0033;">
    <a href="#" class="navbar-brand"
        style="background-color:#FFCC33;color:#000000;border-width:10px;
        border-color:#FFCC33;border-style:solid;padding:6px;border-
radius:5px;">
        Library App
```

```

</a>

<!-- <a class="navbar-link btn-lg" href="{% url 'index' %}"
      style="background-color:#FFCC33;color:#000000;">DataFlair</a>

<a href="{% url 'admin:index' %}" class="navbar-link btn-lg"
      style="background-color:#FFCC33;color:#000000;">Admin</a>

<a href="{% url 'upload-book' %}" class="navbar-link btn-lg"
      style="background-color:#FFCC33;color:#000000;">Upload Books</a>!-->

<a href="{% url 'login' %}" class="navbar-link btn-lg"
      style="background-color:#FFCC33;color:#000000;">Login</a>

<!--<a href="{% url 'dashboard' %}" class="navbar-link btn-lg"
      style="background-color:#FFCC33;color:#000000;">Dashboard</a> !-->
</nav>

<br><br>

<!--LOGIN FORM CARD -->
<div class="container">
  <div class="row justify-content-center">
    <div class="col-md-5">

      <div class="card shadow-lg">
        <div class="card-header text-center" style="background-
color:#FFCC33;">
          <h4>Sign In</h4>
        </div>

        <div class="card-body">

          <!-- Error Message -->
          {% if error %}
            <div class="alert alert-danger text-center">
              {{ error }}
            </div>
          {% endif %}

          <form method="post">
            {% csrf_token %}

            <div class="form-group">
              <label><b>Username</b></label>

```

```

        <input type="text" name="username" class="form-
control" required>

    </div>

    <div class="form-group">
        <label><b>Password</b></label>
        <input type="password" name="password" class="form-
control" required>

    </div>

    <button type="submit" class="btn btn-block"
        style="background-color:#FF0033;color:white;font-
weight:bold;">
        Login
    </button>

</form>
</div>
</div>
</div>
</div>
</body>
</html>

```

Note: This modification ensures that users are redirected to the /dashboard/ URL once they log in successfully.

Code Display:

```

PRACTICE6
> .venv
libraryapp
  > book
    > __pycache__
    > migrations
    > templates\book
      > css
      > Confirm_delete.h...
      > dashboard.html
      > library.html
      > login.html
      > upload_form.html
    > __init__.py
    > admin.py
    > apps.py
    > forms.py
    > models.py
    > tests.py
    > urls.py
    > views.py
  > libraryapp
    > __pycache__
    > __init__.py
    > asgi.py
    > settings.py
    > urls.py
    > wsgi.py
  > media

libraryapp > book > templates > book > login.html
1  <!DOCTYPE html>
2  {% load static %}
3  <html>
4  <head>
5      <title>Library App - Login</title>
6
7      <!-- Bootstrap -->
8      <link rel="stylesheet"
9          href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css"
10         integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T"
11         crossorigin="anonymous">
12
13  </head>
14
15  <body style="background-color:#4DB6AC;">
16
17  <!-- NAVIGATION BAR -->
18  <nav class="navbar navbar-light" style="background-color:#FF0033;">
19      <a href="#" class="navbar-brand"
20          style="background-color:#FFCC33;color:#000000;border-width:10px;
21              border-color:#FFCC33;border-style:solid;padding:6px;border-radius:5px;">
22          Library App
23      </a>
24
25      <!-- <a class="navbar-link btn-lg" href="{% url 'index' %}"
26           style="background-color:#FFCC33;color:#000000;">DataFlair</a>
27
28      <a href="{% url 'admin:index' %}" class="navbar-link btn-lg"
29          style="background-color:#FFCC33;color:#000000;">Admin</a>
30
31      <a href="{% url 'upload-book' %}" class="navbar-link btn-lg"
32          style="background-color:#FFCC33;color:#000000;">Upload Books</a>!-->
33

```

```

34      <a href="{% url 'upload-book' %}" class="navbar-link btn-lg"
35
36      <a href="{% url 'login' %}" class="navbar-link btn-lg"
37          style="background-color:#FFCC33;color:#000000;">Login</a>
38
39      <!--<a href="{% url 'dashboard' %}" class="navbar-link btn-lg"
40           style="background-color:#FFCC33;color:#000000;">Dashboard</a>!-->
41  </nav>
42
43  <br><br>
44  <!-- LOGIN FORM CARD -->
45  <div class="container">
46      <div class="row justify-content-center">
47          <div class="col-md-5">
48              <div class="card shadow-lg">
49                  <div class="card-header text-center" style="background-color:#FFCC33;">
50                      <h4>Sign In</h4>
51                  </div>
52
53                  <div class="card-body">
54
55                      <!-- Error Message -->
56                      {% if error %}
57                          <div class="alert alert-danger text-center">
58                              {{ error }}
59                          </div>
60                      {% endif %}
61
62                      <form method="post">

```

```

62 <form method="post">
63     {% csrf_token %}
64
65     <div class="form-group">
66         <label><b>Username</b></label>
67         <input type="text" name="username" class="form-control" required>
68     </div>
69
70     <div class="form-group">
71         <label><b>Password</b></label>
72         <input type="password" name="password" class="form-control" required>
73     </div>
74
75     <button type="submit" class="btn btn-block"
76         style="background-color:#FF0033;color:white;font-weight:bold;">
77         Login
78     </button>
79 </form>
80 </div>
81 </div>
82 </div>
83 </div>
84 </div>
85 </div>
86 </div>
87 </div>
88 </body>
89 </html>

```

Note: This modification ensures that users are redirected to the /dashboard/ URL once they log in successfully.

6. Creating Logout Functionality

To display the home page/ or login page while the user clicks logout in the dashboard page, we need to add the logout view function, so:

In **views.py**:

Add the logout view to return back to login page:

```

def logout_view(request):
    logout(request)
    return redirect('login') # Redirect to login page after logout

```

Logout will:

- Remove the authentication session
- Return the user to the homepage / Login page

Code Display:

After you add login, dashboard and logout views in *views.py*:

```

libraryapp > book > views.py > logout_view
1  from django.shortcuts import render, redirect
2  from .models import Book
3  from .forms import BookCreate
4  from django.http import HttpResponse
5  from django.contrib.auth.decorators import login_required
6  from django.contrib.auth import authenticate, login, logout
7
8  @login_required
9  def dashboard(request):
10     return render(request, 'book/dashboard.html')
11
12  def login_view(request):
13     if request.method == 'POST':
14         username = request.POST['username']
15         password = request.POST['password']
16
17         user = authenticate(request, username=username, password=password)
18
19         if user is not None:
20             login(request, user)
21             return redirect('dashboard') # Redirect after login
22         else:
23             return render(request, 'book/login.html',
24                             {'error': 'Invalid username or password'})
25
26     return render(request, 'book/login.html')
27
28
29  def logout_view(request):
30     logout(request)
31     return redirect('login') # Redirect to login page after logout
32

```

7. Adding URLs

Now, update *libraryapp/urls.py* to include the login, dashboard and logout URLs:

In:

libraryapp/urls.py

Add the following code:

```

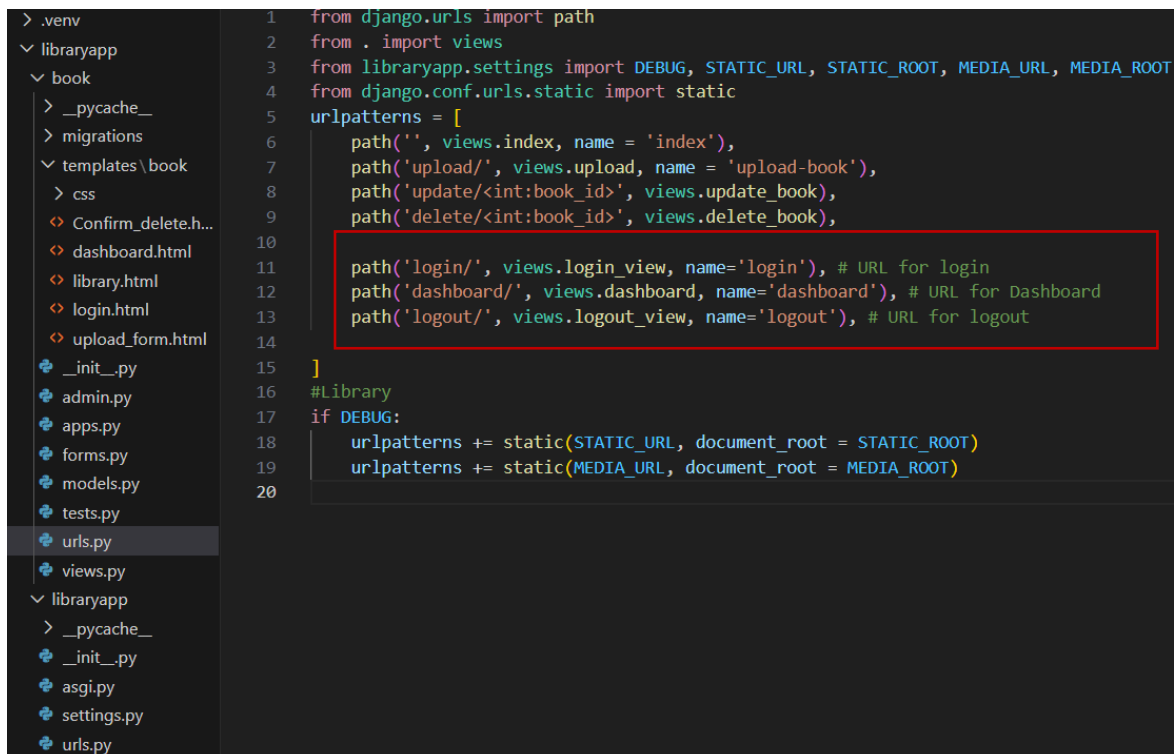
from django.urls import path
from . import views

```

```
urlpatterns = [
.....

    path('login/', views.login_view, name='login'),
    path('dashboard/', views.dashboard, name='dashboard'),
    path('logout/', views.logout_view, name='logout'),
]
```

Code Display:



```
> .venv
✓ libraryapp
  ✓ book
    > __pycache__
    > migrations
    ✓ templates\book
      > css
      > Confirm_delete.h...
      > dashboard.html
      > library.html
      > login.html
      > upload_form.html
    > __init__.py
    > admin.py
    > apps.py
    > forms.py
    > models.py
    > tests.py
    > urls.py
    > views.py
  ✓ libraryapp
    > __pycache__
    > __init__.py
    > asgi.py
    > settings.py
    > urls.py

1 from django.urls import path
2 from . import views
3 from libraryapp.settings import DEBUG, STATIC_URL, STATIC_ROOT, MEDIA_URL, MEDIA_ROOT
4 from django.conf.urls.static import static
5 urlpatterns = [
6     path('', views.index, name = 'index'),
7     path('upload/', views.upload, name = 'upload-book'),
8     path('update/<int:book_id>', views.update_book),
9     path('delete/<int:book_id>', views.delete_book),
10
11     path('login/', views.login_view, name='login'), # URL for login
12     path('dashboard/', views.dashboard, name='dashboard'), # URL for Dashboard
13     path('logout/', views.logout_view, name='logout'), # URL for logout
14 ]
15
16 #Library
17 if DEBUG:
18     urlpatterns += static(STATIC_URL, document_root = STATIC_ROOT)
19     urlpatterns += static(MEDIA_URL, document_root = MEDIA_ROOT)
20
```

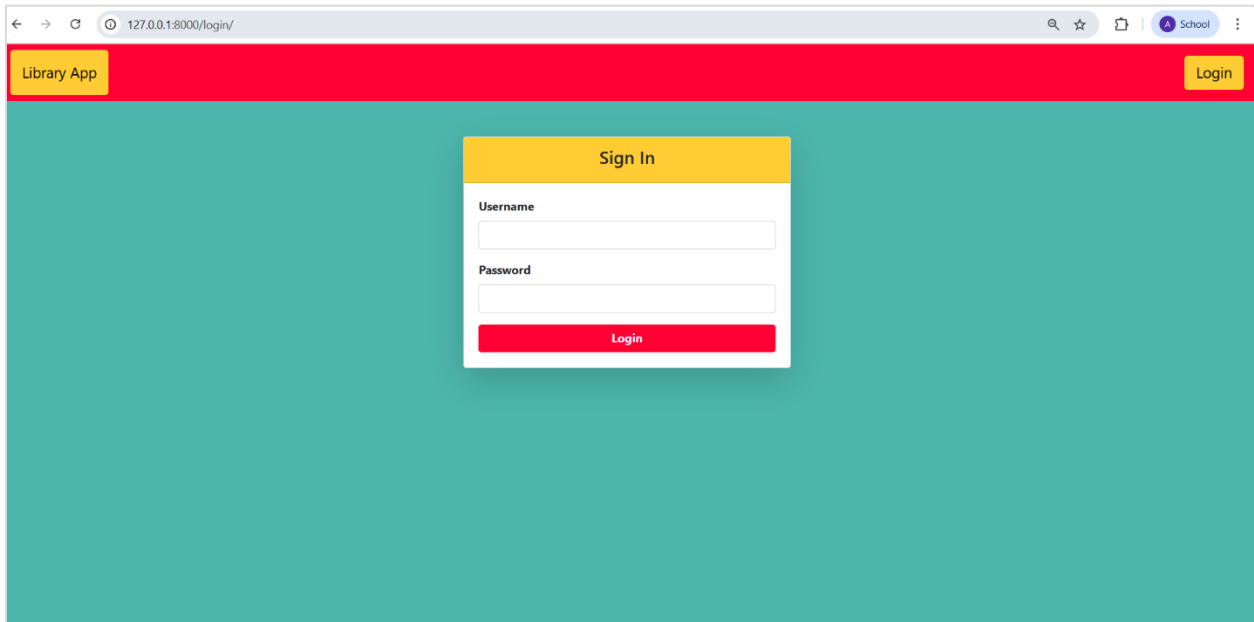
8. Run and Test Your library App

Now, start the server:

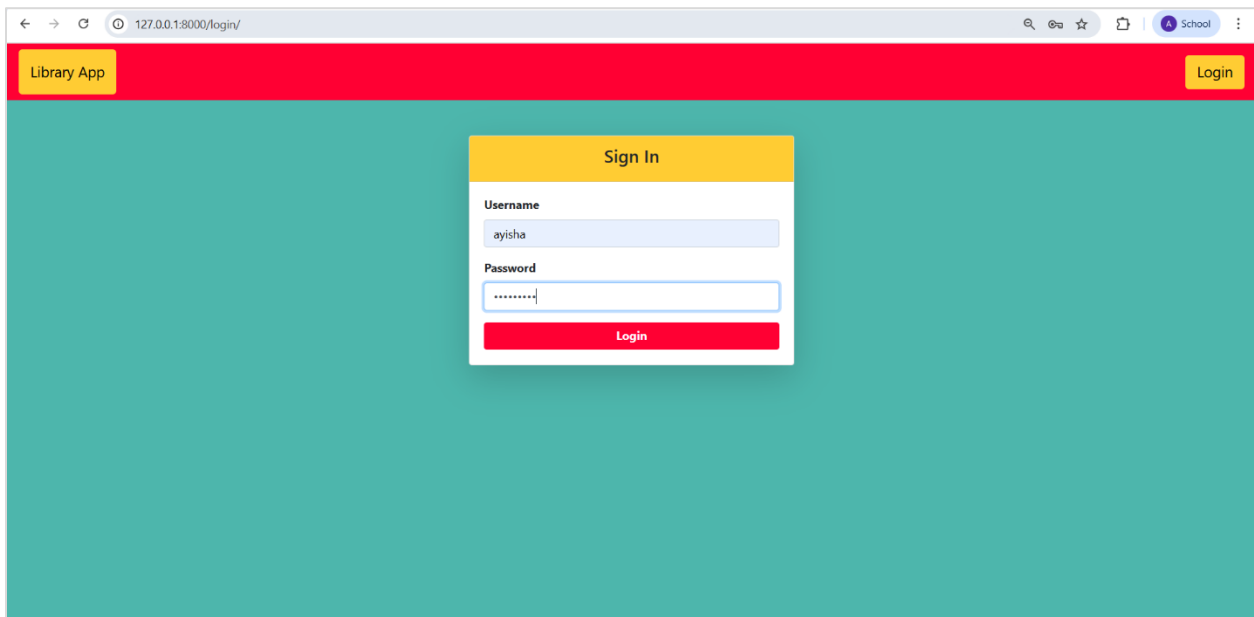
```
python manage.py runserver
```

Now test the app in the browser:

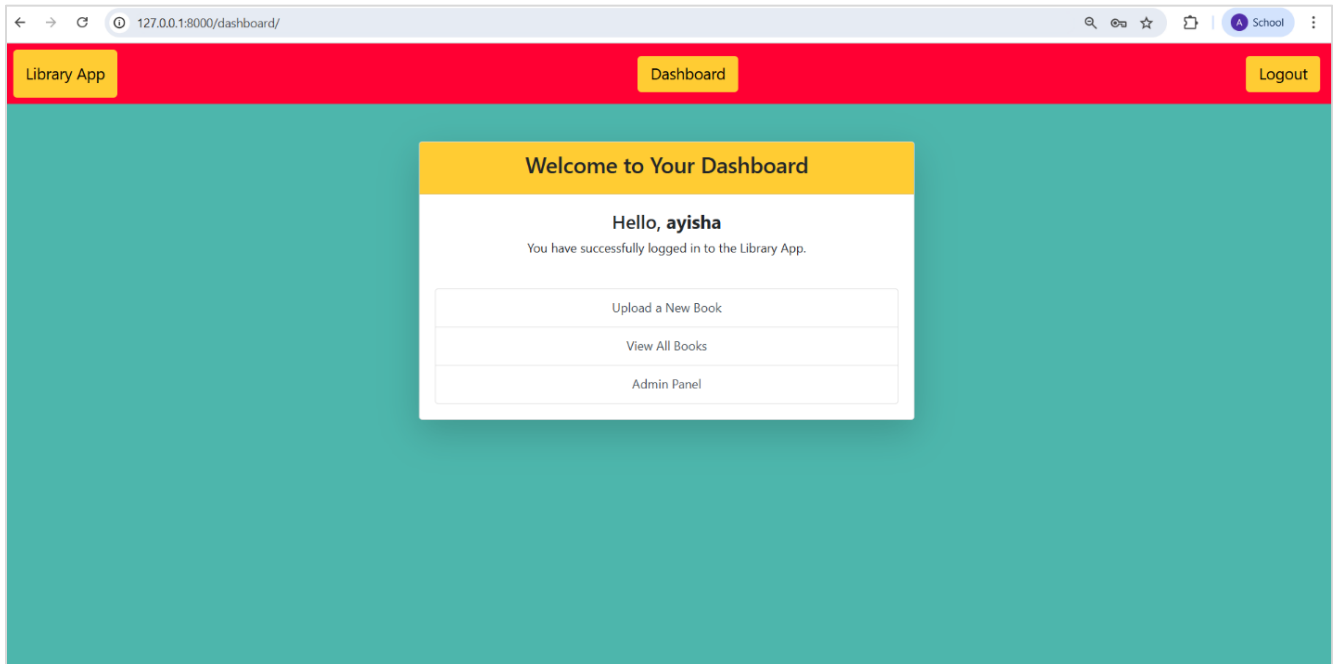
<http://127.0.0.1:8000/login/>



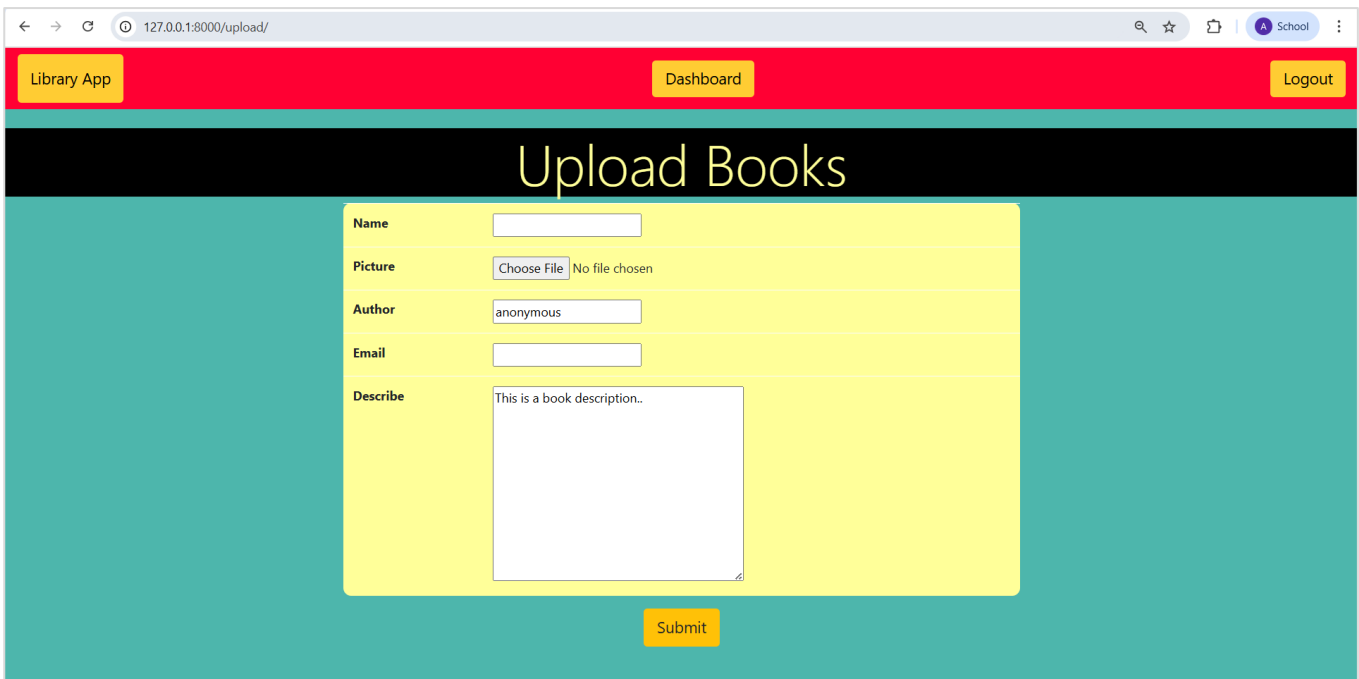
➤ Enter your username and password that you created earlier.



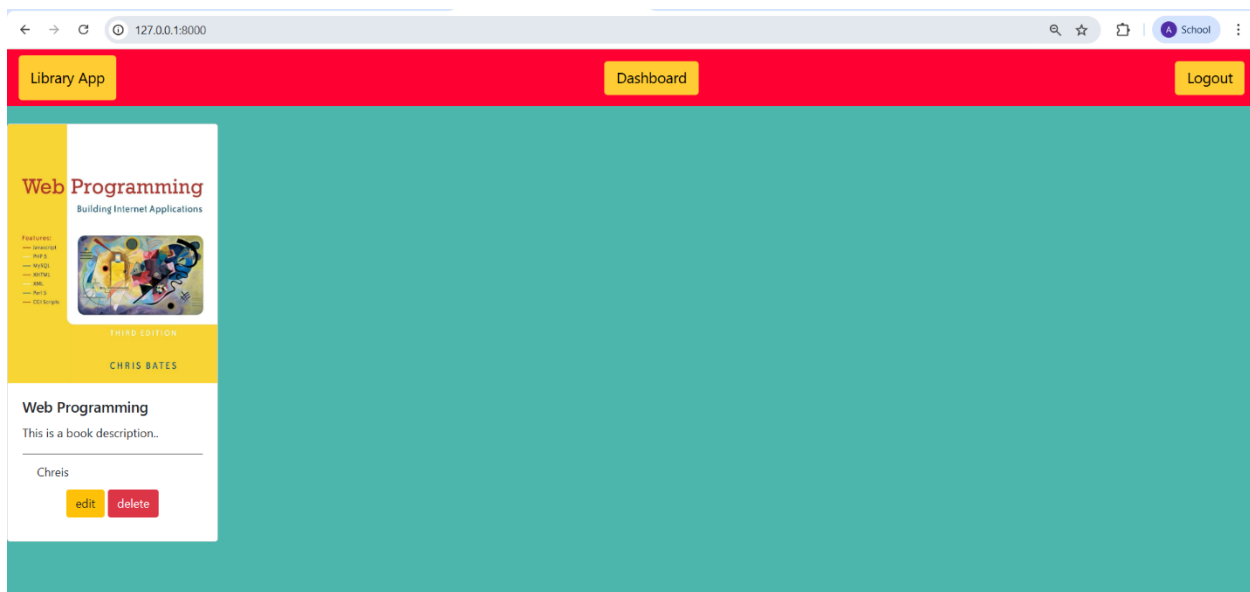
After you login successfully, you will go directly to the **Dashboard**:



➤ Select “*Upload new book*”:



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/upload/'. The page has a red header bar with three buttons: 'Library App', 'Dashboard', and 'Logout'. The main content area has a teal background. At the top of this area is a black banner with the text 'Upload Books' in yellow. Below the banner is a yellow form with the following fields: 'Name' (text input with 'Web Programming'), 'Picture' (file upload button and text 'Web-Programming-4.jpg'), 'Author' (text input with 'Chris'), 'Email' (text input with 'chris@example.com'), and 'Describe' (text area with 'This is a book description..'). A yellow 'Submit' button is located at the bottom right of the form.



- Open *admin Panel*, you will see the books that you have created:
- **Note:** if you face a problem logging into the *admin panel*, then change your password using the command below:

```
python manage.py changepassword admin
```

